

Adaptive Routing for Sensor Networks using Reinforcement Learning

Ping Wang
College of Computer Science
Zhejiang University
Email: wangping_zju@163.com

Ting Wang
Computer Science Department
University of British Columbia
Email: guasars@cs.ubc.ca

Abstract

Efficient and robust routing is central to wireless sensor networks (WSN) that feature energy-constrained nodes, unreliable links, and frequent topology change. While most existing routing techniques are designed to reduce routing cost by optimizing one goal, e.g., routing path length, load balance, re-transmission rate, etc, in real scenarios however, these factors affect the routing performance in a complex way, leading to the need of a more sophisticated scheme that makes correct trade-offs.

In this paper, we present a novel routing scheme, AdaR that adaptively learns an optimal routing strategy, depending on multiple optimization goals. We base our approach on a least squares reinforcement learning technique, which is both data efficient, and insensitive against initial setting, thus ideal for the context of ad-hoc sensor networks. Experimental results suggest a significant performance gain over a naïve Q-learning based implementation.

1. Introduction

Wireless sensor network (WSN) [1, 4] has attracted upon intensive research recently, due to its large range of applications. In most scenarios, sensor networks are modeled as data acquisition systems: a collection of sensor nodes gather environmental information, which is aggregated and routed by other sensors to powerful base stations for processing. This model is illustrated in Fig. 1, where the data packets are relayed from the data sources (dark shaded nodes) to the base station (square node).

Unlike network with powerful nodes and stable links, e.g., Internet, ad-hoc wireless sensor networks feature great dynamics, i.e., unreliable communication, and frequent topology change. Hence it is reasonable to adopt reactive routing protocol that constructs a routing path on demand. Reactive routing enables the following scenario: starting from any node, one selects a successor from the neighbors of current node. By repeatedly jumping from current node

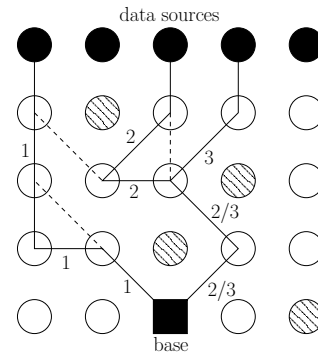


Figure 1. A model of wireless sensor networks. The dark shaded nodes represent data sources, and the square node is the base station. The shaded nodes represent sensor with low residual energy, and the dashed lines indicate unreliable links.

to the successor, the base station is finally reached.

Due to the fact that the sensor nodes are battery-powered devices, prolonging the autonomous lifetime of the network is a challenging problem. In designing energy-efficient routing protocols, the various features of sensor networks lead to a set of optimization problems:

- **Routing path length:** To reduce energy consumption, the collected data needs to be delivered to the base station following the shortest paths, i.e., the routing path length should be minimized;
- **Load balance:** Some sensors residing on the shortest paths of several data sources, can be over-used. One may be interested in conserving these sensors with low residual energy in order to prolong the network lifetime;
- **Link reliability:** The communication between wireless sensors can be quite unstable. One may wish to bypass those unreliable links though they result in a shorter path, because the cost of re-transmission can easily dwarf the save of path length;
- **Aggregation:** The collected data from different sources may be highly correlated. In this case the data can be

aggregated before being delivered to the base station, so that the packet size can be squeezed, thus reducing the transmission cost.

Most existing routing techniques are designed to optimize one of these goals. In real scenarios however, these factors are usually in conflict, and influence the routing performance in a complex way, leading to the need of a more sophisticated routing scheme that makes correct trade-offs. This situation is illustrated in Fig. 1: shaded nodes represent sensors with low residual energy, and dashed lines indicate unreliable communications. All the routing paths approach towards the base station at their best try, however they also take into account other factors. For example, Path 1 bypasses some unreliable links and low energy nodes; Path 2 warps itself in order to combine with Path 3, and the aggregated data is then delivered in a single path. Obviously, solving the optimization goals separately does not lead to an optimal solution of the problem here; instead one needs to consider all these optimization goals as a whole in designing routing protocols.

In this paper, we present a novel routing scheme *AdaR*¹, which enables the sensor nodes as a team to efficiently learn an *optimal* routing strategy, depending on all the optimization goals discussed above (though IT is flexible to be extended). Specifically, we base our approach on Least Squares Policy Iteration (LSPI) [2, 11], a highly efficient model-free reinforcement learning technique, which makes it possible to learn a best strategy with small number of tries and is insensitive against initial parameter setting, thus ideal for the context of ad-hoc sensor networks.

The remainder of the paper will be organized as follows: Section 2 presents a belief survey of related work; In Section 3, we give an overview of reinforcement learning techniques for sensor networks; In Section 4, we describe in detail our routing scheme based on least squares reinforcement learning; The experimental results are shown in Section 5; The paper is ended with a conclusion and future work in Section 6.

2. Related Work

The various features of wireless sensor networks lead to a set of optimization problems in designing energy-efficient routing protocols. Most existing routing techniques are designed to optimize one of these goals. For example, ‘GPSR’ [9] and [7] are proximity-based routing strategies that aim at finding the shortest routing path; R. Shah et al [14] proposed a routing scheme that takes into account the residual energy of sensor nodes, in order to balance the load; ‘Directed Diffusion’ [8] is a fault tolerant routing strategy that is robust against unstable communication.

¹“AdaR” stands for Adaptive Routing.

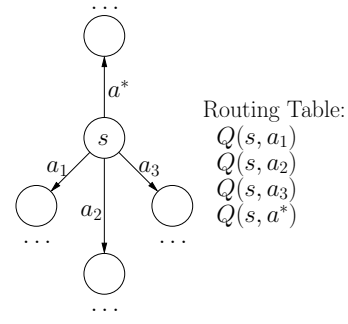


Figure 2. The routing table (Q-values) of a sensor s . The action a^* with the maximum Q-value is selected.

To the best of our knowledge, no existing routing scheme takes in consideration all these optimization goals together.

The idea of applying reinforcement learning to routing in networks was firstly introduced by Boyan and Littman [3], who used Q-learning [15, 16] in static packet switched network. They have shown that the Q-learning based routing approach is able to compete with the shortest path algorithms, without prior knowledge regarding the network topology. Following their prior work, Q-learning is applied to routing in ad-hoc networks [6] and optimizing the trade-off between routing and compression [12].

3. Preliminaries

3.1. RL-based routing

Reinforcement learning (RL) [15] provides a framework by which an agent can learn control policies based on experience and rewards. The underlying concept of RL is that of Markov Decision Process (MDP).

An MDP models an agent acting in an environment with a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is a set of actions. $\mathcal{P}(s'|s, a)$ is the transition model that describes the probability of entering state s' after executing action a at state s . $\mathcal{R}(s, a, s')$ is the reward obtained when the agent executes a at s and enters s' . Within this framework, the quality of action a at state s can be denoted by a Q-value $Q(s, a)$.

The goal of solving an MDP is to find an optimal policy, $\pi : \mathcal{S} \mapsto \mathcal{A}$, that maps states to actions such that the cumulative reward is maximized, which is equivalent to finding the following Q-fixed points (Bellman equation):

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s'} \mathcal{P}(s'|s, a) \max_{a'} Q^*(s', a') \quad (1)$$

where the first term on the right side represents the expected immediate reward of executing action a at s , which is calculated as $r(s, a) = \sum_{s'} \mathcal{P}(s'|s, a) \mathcal{R}(s, a, s')$, and the second

term is the maximum expected future reward. The discount factor γ models the fact that immediate reward is more valuable than future reward.

In the context of learning routing strategy, each sensor can be considered as a state s , and for each of its neighbor s' , there is a corresponding action a ($\mathcal{P}(s'|s, a) = 1$). Executing action a at s means forwarding the packet to the corresponding neighbor s' . The fixed Q-values of s can be regarded as its routing table, telling to which neighbor to forward the data. This scheme is illustrated in Fig. 2. With the help of this routing table, the optimal routing path can be trivially constructed by a sequence of table look-up operations. Thus the task of learning optimal routing strategy is equivalent to finding the Q-fixed points (Equation 1).

3.2. Q-learning

If the underlying transition model $\mathcal{P}$ and reward mechanism $\mathcal{R}$ is known, the Q-fixed points can be solved deterministically. In most cases, such as ad-hoc sensor networks however, it is fairly hard to determine the underlying model, which calls for stochastic reinforcement learning techniques.

Q-learning is an off-policy temporal difference (TD) control algorithm, that directly approximates the optimal action-value function (Equation 1). Each time an action a is executed, the agent receives an immediate reward r from the environment. It then uses this reward and the expected long term reward to update the Q-values, which in turn influences future action selection. Its simplest form, one-step Q-learning is defined as:

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')]$$

where α is the learning rate, which models the rate of updating Q-values.

As a model-free RL technique, Q-learning requires no knowledge about the underlying reward or transition mechanism, thus applicable to the problem of learning routing strategy in sensor networks.

4. AdaR Approach

In this section, we describe in detail our AdaR routing scheme. We first present an analysis of the drawbacks of directly applying Q-learning to learning optimal routing strategy. Then we introduce Least Squares Policy Iteration (LSPI), a newly proposed reinforcement learning technique, and show its feasibility for our purpose. Based on LSPI, our AdaR routing scheme is constructed.

4.1. Naïve Q-learning implementation

Though Q-learning has been applied widely to routing in ad-hoc networks [3, 12, 6], due to its simplicity, our preliminary tests show that a naïve Q-learning implementation does have several severe drawbacks:

- It requires a large number of tries before being able to converge to the optimal solution. In the scenario of sensor network, the cost of sampling may easily dwarf the gain of finding an optimal path;
- It is quite sensitive to the parameter setting, i.e., small adjustments can result in serious fluctuation in routing performance;
- As indicated in [12], introducing basic Q-learning into the setup of sensor networks may violate some necessary convergence conditions of MDP.

4.2. LSPI

To overcome the problems above, we apply Least Squares Policy Iteration (LSPI), a recently introduced reinforcement learning technique (detail is referred to [11]). Instead of evaluating the optimal action-value function (Equation 1) directly, LSPI approximates Q-values Q^π for a given policy π with a parametric function approximator. More precisely, the value function is approximated as a linear weighted combination of k basis functions (features):

$$\hat{Q}^\pi(s, a, w) = \sum_{i=1}^k \phi_i(s, a)w_i = \phi(s, a)^T w \quad (2)$$

where $\phi_i(s, a)$ is the i th basis function, and w_i is its weight in the linear equation. The k basis functions (features) represent the information we know about each state-action pair, which are designed manually.

Equation 1 can then be re-written as $\Phi w \approx \mathcal{R} + \gamma \mathcal{P}^\pi \Phi w$, where Φ is a $(|S||A| \times k)$ matrix, representing the basis functions for all state-action pairs. Assuming that the columns of Φ are linearly independent, this equation can be reformulated as: $\Phi^T(\Phi - \gamma \mathcal{P}^\pi \Phi)w^\pi = \Phi^T \mathcal{R}$

The weights w of the linear functions $\hat{Q}^\pi(s, a)$ can be extracted by solving the following linear system:

$$\begin{aligned} w &= A^{-1}b \\ \text{where } A &= \Phi^T(\Phi - \gamma \mathcal{P}^\pi \Phi) \\ b &= \Phi^T \mathcal{R} \end{aligned} \quad (3)$$

As a model-free learning algorithm, LSPI learns A and b by sampling from the environment. A sample is a tuple (s, a, s', r) , where s, a, s' and r are the current state, action, new state, and immediate reward respectively. Given a set of samples, $D = \{(s_{d_i}, a_{d_i}, s'_{d_i}, r_{d_i}) | i = 1, 2, \dots, L\}$, one

can construct approximate version of Φ , $\mathcal{P}^\pi \Phi$ and $\mathcal{R}$ as:

$$\hat{\Phi} = \begin{pmatrix} \phi(s_{d_1}, a_{d_1})^T \\ \dots \\ \phi(s_{d_L}, a_{d_L})^T \end{pmatrix} \quad \widehat{\mathcal{P}^\pi \Phi} = \begin{pmatrix} \phi(s'_{d_1}, \pi(s'_{d_1}))^T \\ \dots \\ \phi(s'_{d_L}, \pi(s'_{d_L}))^T \end{pmatrix}$$

$$\hat{\mathcal{R}} = \begin{pmatrix} r_{d_1} \\ \dots \\ r_{d_L} \end{pmatrix}$$

With $\hat{\Phi}$, $\widehat{\mathcal{P}^\pi \Phi}$ and $\hat{\mathcal{R}}$, the weight w can then be solved using Equation 3.

We argue that LSPI is suitable for our purpose for the following reasons:

- LSPI converges faster with less samples than naive Q-learning, since the samples are used more efficiently. This property comes from the fact that LSPI evaluates the policy with a single pass over the set of samples, and all the samples can be used in each iteration to evaluate the policy.
- LSPI does not require carefully tuning initial parameters, e.g., learning rate. This property comes from the fact the policy is evaluated over a whole history of sampling, thus insensitive to initial parameter setting;
- The assumption that Q-values can be approximated by a linear architecture of features is suitable in our setting. As indicated above, the value of selecting a neighbor to which to forward the data depends on a set of optimization goals, and a linear combination is expressive to represent the trade-off between these goals.
- While naive Q-learning makes decision directly based on Q-values, which may be lagged behind the dynamics of the network, LSPI learns the weights of the linear functions, thus can update the Q-values based on the most updated information regarding the features.

4.3. AdaR

In our routing scheme AdaR, we apply LSPI to the problem of learning an optimal routing strategy. For a sensor node s and forwarding action a , let s' be the corresponding neighbor node ($\mathcal{P}(s'|s, a) = 1$). We consider the following features for the state-action pair (s, a) : (1) the difference of the distances (hop count) of s and s' to the base station, $d(s, a)$; (2) the residual energy of s' , $e(s, a)$; (3) the number of routing paths crossing on s' , $c(s, a)$, which is extracted from the sampling procedure; (4) the reliability of the link between s and s' , $l(s, a)$. Formally, the collection of basis functions for (s, a) is $\phi(s, a) = \{d(s, a), e(s, a), c(s, a), l(s, a)\}$. Note that all these values have been normalized to the range of $[-1, 1]$.

The AdaR approach samples from the environment a set of tuples (s, a, s', r) , and use LSPI to update the weights

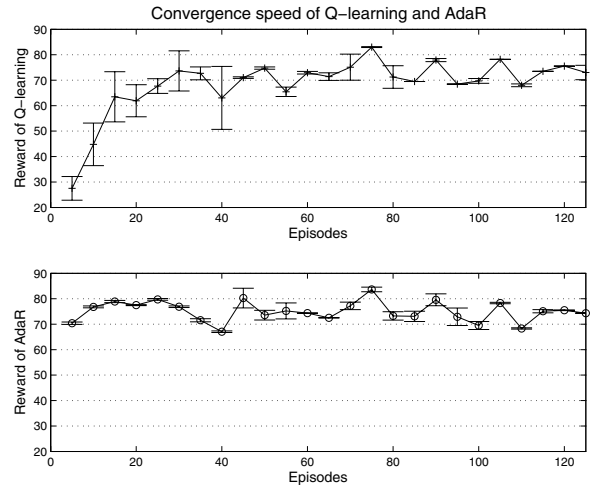


Figure 3. Convergence speed of naïve Q-learning and AdaR. Cumulative reward is calculated for all the routing paths at each episode.

w of the linear functions $\hat{Q}^\pi(s, a)$ for the current policy π . The updated weights in turn are used to improve the current policy π . This procedure repeats with the improved policy, until a fixed point policy is reached, i.e., the weights of policies between successive iterations do not differ significantly.

Specifically, the learning procedure runs as follows: when a packet is forwarded from node s to s' by action a , based on the current Q-values $\phi(s, a)^T w$, a record $\langle s, a, s', \phi(s, a) \rangle$ is appended to the packet. Thus on the packet arriving at the base station, one can trace the information of the whole routing path. The immediate reward r for each tuple (s, a, s') is then calculated based on the quality of the routing path, which will be discussed in more detail in next section. Note that the routing may get trapped in a circle, in which case a small-scale flooding is usually sufficient. After a certain amount of samples (s, a, s', r) is collected at the base station, one invokes the LSPI procedure to estimate new weights w' of the linear functions $\hat{Q}^\pi(s, a)$.

The updated w' can then be broadcasted within the network, or be piggybacked when a query is disseminated from the base station. With the updated weights w' , policy improvement is done easily at each sensor s by selecting the action a with the highest Q-value: $\pi(s|w') = \arg \max_a \phi(s, a)^T w'$. This procedure repeats with the improved policy, until reaching a fixed point policy.

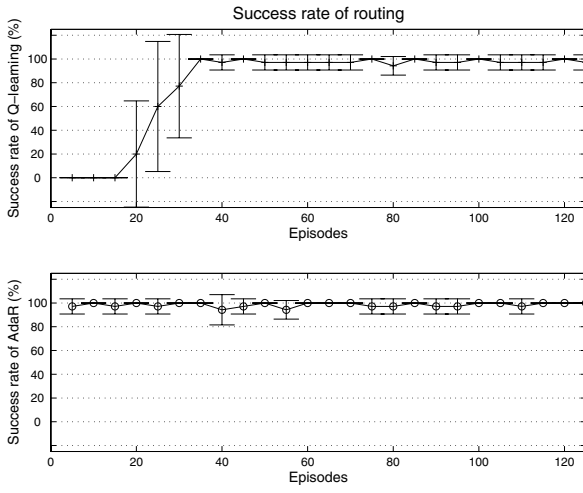


Figure 4. Success rate (%) of routing of naïve Q-learning and AdaR.

5. Experiments

In this section, we present an experimental analysis of our AdaR approach with mainly two aims: (1) To evaluate the convergence speed of AdaR, we studied the number of iterations it takes to converge to a fixed policy and the quality of resulting routing strategy; (2) To evaluate the sensitivity of our approach to the initial parameters, we run the algorithm under different settings. In both cases, we compared our approach with a naïve Q-learning implementation.

5.1. Experimental setting

The simulated network consists of 400 sensors, deployed on a perturbed grid over a square with length 80 units. Each sensor can communicate with neighbors within a radius of 15 units. One base station is deployed at the rim of the network. Of the 400 sensors, 20 nodes are randomly selected as data sources. The residual energy of sensor nodes, and the reliability of links are both randomly distributed in the range of $(0, 1)$. To simulate the dynamics of the network, we periodically change the residual energy and link reliability following a Markov model.

At the beginning of a training episode, each data source initiates a packet. At each step, one neighbor s' of current node s is selected as the successor, based on current Q-values $Q(s, a) = \phi(s, a)^T w$. To speedup exploration, an ϵ -greedy mechanism is applied, i.e., with probability $1 - \epsilon$, one takes the action with the maximum Q-value, and with probability ϵ one chooses an action at random. Each step generates a record of $\langle s, a, s', \phi(s, a) \rangle$. The package is repeatedly forwarded from the current node to its successor, towards the base station. An episode completes if for each

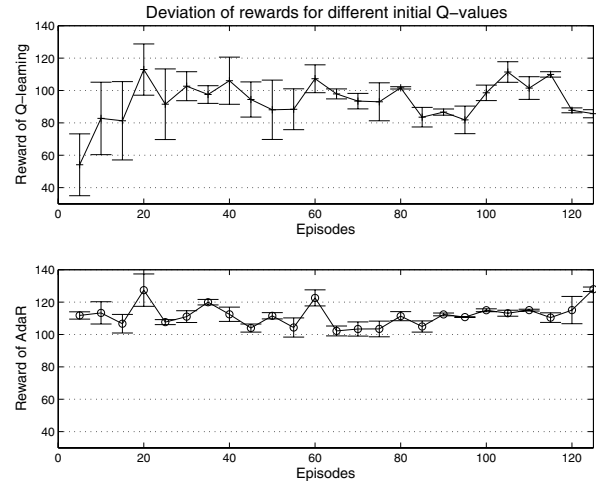


Figure 5. Deviation of cumulative rewards for different initial Q-values.

path one of the following events happens, and the path is given a reward R :

1. The base station is reached. In this case, a positive reward is given: $R = R_{max}$;
2. The path length (hop count) L exceeds a threshold L_{max} , but the base station is not reached. In this case, a neutral reward is given: $R = 0$;
3. A cycle appears in the path. A negative reward is given for this case: $R = -R_{max}/2$.

The immediate reward r for each tuple (s, a, s') is calculated as the reward of whole path divided by the path length, plus a combination of the features. Formally, for a tuple (s, a, s', r) , $r = R/L + \phi(s, a)^T \omega$, where ω is a weight vector. Note that this linear combination here is optional, and more complex model of reward mechanisms can be used instead, e.g., the immediate reward decreases exponentially as the residual energy. The default setting of parameters are as follows: learning rate $\alpha = 0.5$, discount factor $\gamma = 0.5$, exploration rate $\epsilon = 0.1$, and weight parameter ω is a vector of all 1s.

5.2. Convergence speed

The comparison of the convergence speed of both routing approaches is shown in Fig. 3 and Fig. 4. At each episode, we measured the cumulative rewards of all the routing paths and the success rate of arriving at the base station, for AdaR and naïve Q-learning implementations. It can be noticed that under both metrics AdaR takes significantly less iterations to reach the best trade-off than the naïve Q-learning approach, which is especially evident in the plot of success rate of routing. Since each iteration/episode takes same amount of time in both cases, one

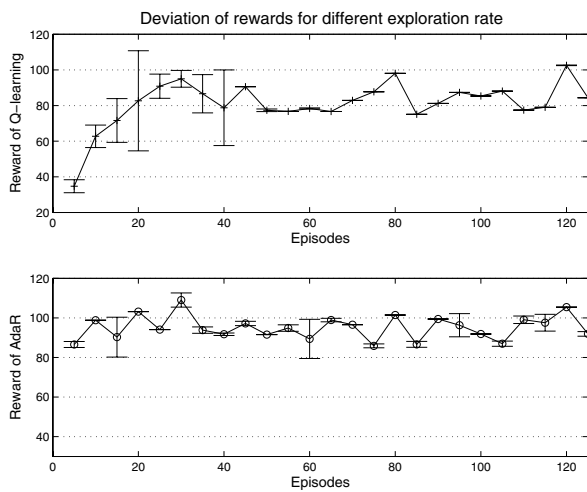


Figure 6. Deviation of cumulative rewards for different exploration rate ϵ .

can conclude that the AdaR approach converges much faster than the naïve Q-learning one. These results empirically approved our analysis that LSPI uses data more efficiently than Q-learning.

From the plots of cumulative reward and success rate of routing, one can also notice that the quality of the optimal routing strategies found by AdaR is as good as, if not better than that by the naïve Q-learning implementation.

5.3. Sensitivity against setting

To study the sensitivity of both approaches to the initial setting, we ran the simulation with different initial parameters. Specifically, we measured the cumulative rewards of all routing paths achieved by both approaches, for different initial Q-values, and different exploration rate ϵ . The results are plotted in Fig. 5 and 6 respectively. It is clear that the performance of AdaR is more stable than that of naïve Q-learning. This property can be contributed to the fact LSPI uses the whole history of samples to evaluate the policy, thus insensitive against initial setting.

In our experiments, we also noticed that both naïve Q-learning and AdaR are quite sensitive to the learning rate α . One solution to this problem is to use Bayesian exploration [5], which will be explored in our future work.

6. Conclusion and Future Work

In this paper, we present a novel routing scheme *AdaR* for wireless sensor networks, which can make correct trade-offs between multiple optimization goals in order to maximize network lifetime. The approach is based on a least squares reinforcement learning technique, which is both

data efficient and insensitive against initial setting. The experimental results suggest a significant performance gain over a naïve Q-learning implementation.

Following some prior work on machine-learning-based routing for wireless sensor networks, this paper explores further the possibility of learning highly efficient routing strategies with multiple optimization goals. Some future work includes applying AdaR to real testbed to verify its feasibility, adapting this learning approach to other routing models of sensor networks, e.g., point-to-point routing, and extending the technique to the case that multiple sensors cooperate with each other to facilitate routing.

References

- [1] I. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci. A survey on sensor networks. *IEEE Commun. Mag.*, 40:102–114, 2002.
- [2] J. Boyan. Least-squares temporal difference learning. In *Proc. of ICML*, 1999.
- [3] J. Boyan and M. Littman. Packet routing in dynamically changing networks: A reinforcement learning approach. In *Proc. of NIPS*, 1994.
- [4] D. Culler, D. Estrin, and M. Srivastava. Overview of sensor networks. *IEEE Computer*, pages 41–49, 2004.
- [5] R. Dearden, N. Friedman, and D. Andre. Model based bayesian exploration. In *Proc. of UAI*, 1999.
- [6] M. Fakir. Resource optimization methods for telecommunication networks. 2004.
- [7] J. Gao, L. Guibas, J. Hersherberger, L. Zhang, and A. Zhu. Geometric spanner for routing in mobile networks. In *Proc. of ACM MobiHoc*, 2001.
- [8] C. Intanagonwiwat, R. Govindan, and D. Estrin. Directed diffusion: a scalable and robust communication paradigm for sensor networks. In *Proc. of ACM MobiCom*, 2000.
- [9] B. Karp and H. Kung. Gpsr: greedy perimeter stateless routing for wireless networks. In *Proc. ACM MobiCom*, 2000.
- [10] C. Kwok and D. Fox. Reinforcement learning for sensing strategies. In *Proc. of IROS*, 2004.
- [11] M. Lagoudakis and R. Parr. Model-free least-squares policy iteration. In *Proc. of NIPS*, 2001.
- [12] K. S. P. Beyens, M. Peeters and A. Nowe. Routing with compression in wireless sensor networks: a q-learning approach. In *Proc. of AAMAS*, 2005.
- [13] E. Royer and C. Toh. A review of current routing protocols for ad-hoc mobile wireless networks. *IEEE Personal Communications*, pages 46–55, 1999.
- [14] R. Shah and J. Rabaey. Energy aware routing for low energy ad hoc sensor networks. In *Proc. of WCNC*, 2002.
- [15] R. Sutton and A. Barto. *Reinforcement Learning: An introduction*. MIT Press, Cambridge, MA, 1992.
- [16] C. Watkins and P. Dayan. Q-learning. *Machine Learning*, pages 279–292, 1992.